Intelligent Threat Qualification Engine (ITQE): An Integrative Industry Synthesis for Automated SOC Triage
Module Summary

Author: Aung Ze Ya | **Date:** November 7, 2026 | **Course:** MSc in Artificial Intelligence
Primary Frameworks: Python | Scikit-Learn | PyTorch | Hugging Face Transformers | Plan-and-Execute | Jupyter Notebook

Overview

Modern Security Operations Centers (SOCs) face a critical scalability crisis driven by alert fatigue. Tier-1 analysts are overwhelmed by thousands of daily alerts, forcing them to manually translate low-level network telemetry into readable incident reports. This manual triage delays response times and introduces human error. While automated machine learning models can classify network traffic with high accuracy, they often output raw classification labels that a non-specialist cannot interpret, prioritize, or act upon (Alahmadi et al., 2022).

Generative AI offers a promising solution by translating raw telemetry into natural-language briefs. However, deploying Large Language Models (LLMs) in cybersecurity introduces a new danger: *automation bias*. LLMs are prone to hallucination, producing grammatically fluent but numerically incorrect summaries (Ji et al., 2023). If a Tier-1 analyst treats an LLM-generated brief as ground truth, they may initiate mitigations based on fabricated data.

This project addresses this industry problem by designing and implementing the **Intelligent Threat Qualification Engine (ITQE)**. ITQE is an integrated AI system that synthesizes methods from three prior capstone projects. It ingests raw network telemetry, uses a trained Random Forest classifier (Project 3) to filter benign traffic, routes confirmed attacks to a fine-tuned DistilGPT2 model (Project 5) to draft a brief, and uses a bounded Plan-and-Execute agent (Project 6) to verify the generated text before it reaches a human analyst.

Integration Rationale

The ITQE system is not a collection of loosely related components; it is an intentional, sequential synthesis of three distinct AI domains. Each prior project contributes a critical layer to the final system:

- **Project 3 (Applied Machine Learning):** Provides the `ThreatClassifier`. This Random Forest model acts as the first gate, achieving high accuracy **(99.66%)** and high precision **(99.87%)** on the **NSL-KDD benchmark.** Its role in ITQE is to filter out benign traffic, reducing the computational burden on the LLM by preventing it from drafting briefs for normal traffic.
- **Project 5 (Generative AI):** Provides the `BriefWriter`. This locally hosted, fine-tuned DistilGPT2 model translates tabular network telemetry into structured, professional SOC advisory briefs. However, Project 5's auditing revealed an 80% failure rate in copying numerical telemetry metrics.
- **Project 6 (Agentic AI):** Provides the `InputGate` and the bounded Plan-and-Execute state machine. Project 6 established the principle of bounded autonomy, explicitly outputting "cannot verify" when tooling cannot confirm a result.

The New Synthesis Layer (Project 7): Because Project 5 proved that the LLM cannot be trusted to copy numbers accurately, and Project 6 taught us to bound AI authority, Project 7 introduces a novel `SlotVerifier`. This programmatic layer sits between the LLM output and the human analyst, cross-referencing the generated English text against the raw tabular data. If numerical drift is detected, the agent forces a regeneration; if drift persists, it refuses to certify the brief, routing it to a human.

By integrating these three projects, ITQE creates a system that is greater than the sum of its parts: it filters noise (P3), translates telemetry (P5), orchestrates the workflow (P6), and guarantees numerical fidelity (P7).

System Design and Technical Decisions

The ITQE architecture operates as a single-agent, semi-autonomous workflow. A single agent was chosen over a multi-agent system because the SOC triage workflow is strictly linear and state-dependent. Multi-agent coordination would introduce unnecessary handoff overhead and failure modes; a single state machine maintains a cohesive findings store and deterministic execution path.

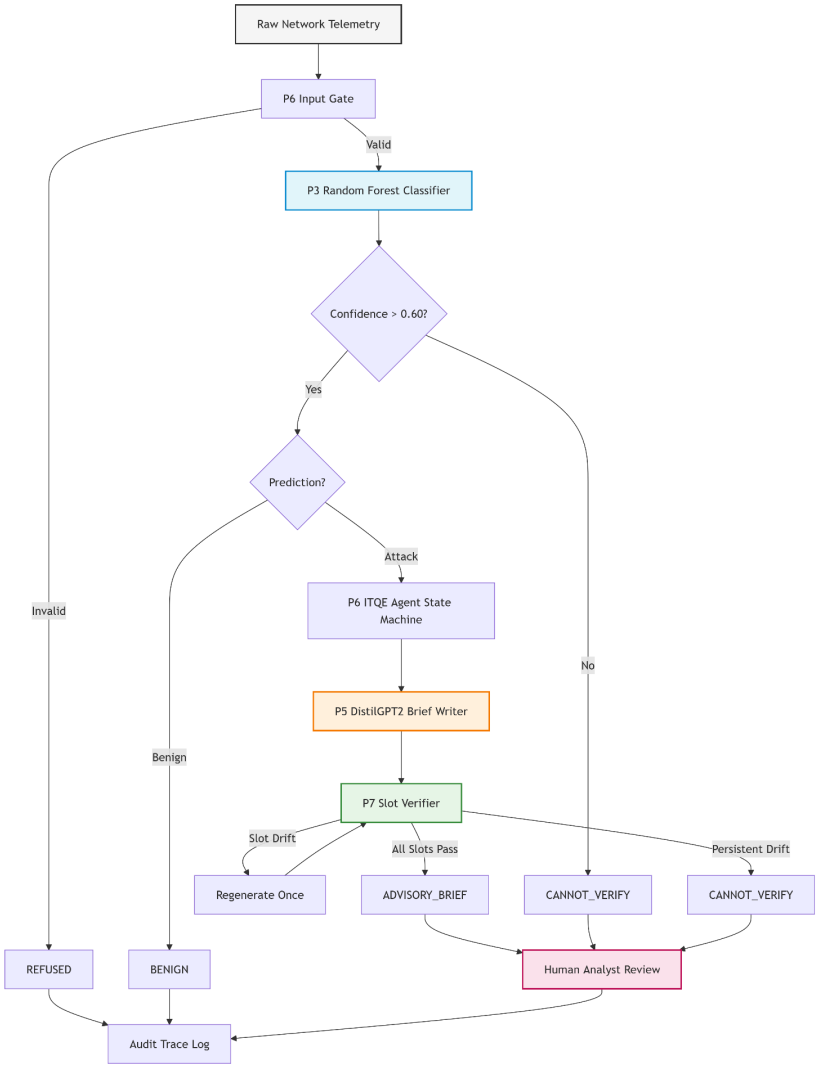


Figure 1: ITQE System Architecture. Raw telemetry passes through an input gate (P6), a Random Forest classifier (P3), a DistilGPT2 brief writer (P5), and finally a Slot Verifier (P7) before routing to a human analyst or the audit log.

The system processes telemetry through four sequential gates:

1. **Input Gate (P6):** The agent validates the input schema. If fields are missing or non-numeric (e.g., `src_bytes: "abc"`), the agent refuses the input at Iteration 0, preventing wasted compute.
2. **Classifier Gate (P3):** The Random Forest model classifies the traffic. If the prediction is `BENIGN`, the agent terminates immediately. If it is an `ATTACK`, the agent checks the confidence score. If confidence is below 0.60, the agent outputs `CANNOT_VERIFY` and routes the event to human review, avoiding premature automation.
3. **Draft Gate (P5):** Confirmed attacks are passed to the DistilGPT2 model. The model generates an English incident brief using a specialized prompt template.
4. **Verification Gate (P7):** The `SlotVerifier` extracts critical numerical fields (`serror_rate`, `rerror_rate`,

`src_bytes`, `dst_bytes`) from the raw telemetry and checks if they are present in the generated text using regular expressions. If slots are missing or altered, the agent triggers a re-check (generating the brief one more time). If the re-check fails, the agent outputs `CANNOT_VERIFY`, and the draft is rejected.

Python

```python
class SlotVerifier:
    def __init__(self):
        self.critical_slots = ['serror_rate', 'rerror_rate', 'src_bytes', 'dst_bytes']

    def verify(self, telemetry: dict, generated_brief: str) -> dict:
        results = {}
        for slot in self.critical_slots:
            expected_val = telemetry.get(slot)
            if 'rate' in slot:
                expected_str = f"{float(expected_val):.2f}"
            else:
                expected_str = str(int(float(expected_val)))

            # \b ensures 0.95 does not match 0.950
            pattern = r'\b' + re.escape(expected_str) + r'\b'
            if re.search(pattern, generated_brief):
                results[slot] = {"status": "PASS", "expected": expected_str}
            else:
                results[slot] = {"status": "FAIL", "expected": expected_str}
        return results
```

This design enforces strict boundaries. The agent never executes network mitigations, never blocks IPs, and never certifies an incident without verification. The human analyst retains full authority over remediation.

Ethical Considerations and Responsible AI

Deploying GenAI in cybersecurity requires addressing several critical ethical and operational risks.

Automation Bias and the "Gibberish" Problem: As demonstrated in the ITQE evaluation, the fine-tuned DistilGPT2 model occasionally collapsed into semantically incoherent text (e.g., "Critical-severalDoN event (Neps)") when given out-of-distribution telemetry. If this text reached a human analyst without safeguards, it would cause confusion and delay. The ITQE architecture mitigates this through its `CANNOT_VERIFY` safeguard. The system does not output a `ADVISORY_BRIEF` unless the numerical data is verified, forcing human-in-the-loop (HITL) review for any ambiguous or drifted outputs (National Institute of Standards and Technology, 2023).

Silent False Negatives: During evaluation, ITQE's Random Forest classified a Loadmodule malware attack as `BENIGN` with low confidence. Because the agent accepted this classification, it terminated without drafting a brief. This is a **silent false negative**—a dangerous failure mode where the system appears to work correctly while silently missing an attack. This limitation is inherited from the NSL-KDD dataset's class imbalance (Tavallaee et al., 2009). ITQE cannot fix this via agentic logic; it must be addressed at the data layer via oversampling or cost-sensitive learning. This underscores that AI systems must be continuously monitored and retrained, and human analysts must never be fully removed from the loop.

Data Privacy: Because the P5 model was trained on network telemetry, there is a risk of training data reconstruction attacks. ITQE mitigates this by running the fine-tuned model entirely locally (on CPU/GPU) rather than through a third-party cloud API, ensuring proprietary enterprise telemetry never leaves the local environment.

Evaluation and Reflection

ITQE was evaluated on five distinct fixtures: a Neptune DoS attack (F1), a Satan Probe (F2), a normal HTTP connection (F3), malformed telemetry (F4), and a Loadmodule malware attack (F5).

Fixture	Input Type	Classification	Verification	Final Status	What It Proves
F1	Neptune DoS	ATTACK (1.00)	FAIL (Slot Drift)	CANNOT_VERIFY	P7 Verifier catches LLM hallucination.
F2	Satan Probe	ATTACK (1.00)	FAIL (Slot Drift)	CANNOT_VERIFY	Verifier catches drift across multiple attack types.
F3	Normal HTTP	BENIGN (0.01)	Skipped	BENIGN	Confidence gate saves LLM compute.
F4	Malformed	*N/A*	Skipped	REFUSED	P6 Input Gate rejects bad schema (Iteration 0).
F5	Loadmodule	BENIGN (0.33)	Skipped	BENIGN	Silent False Negative (Class Imbalance).

What Worked:
The system successfully demonstrated bounded autonomy. On F1 and F2, the Random Forest correctly identified the attacks with 100% confidence. The LLM drafted a brief, but the P7 Slot Verifier caught severe numerical drift (the LLM hallucinated the `serror_rate` and `src_bytes`). The agent forced a regeneration, the drift persisted, and the system correctly routed the event to `CANNOT_VERIFY`. Without the P7 Verifier, a hallucinated brief would have been passed to an analyst. On F3, the confidence gate terminated the process early, saving LLM compute. On F4, the Input Gate rejected the malformed data before any model ran.

What Did Not Work:
The system failed to detect the Loadmodule attack (F5). The Random Forest classified it as `BENIGN`, and the agent accepted this. This is a *silent false negative*, demonstrating that the agent cannot verify what the underlying ML model does not see. Furthermore, the LLM's semantic collapse on real telemetry (generating gibberish) highlights the limitations of an 82M-parameter model. While the Verifier caught the numbers drifting, it could not verify if the English sentence made sense.

Reflections:
The most important lesson from this synthesis is that agentic orchestration is only as good as the tools it orchestrates. A bounded agent can prevent runaway loops and enforce verification, but it cannot fix a biased classifier or a hallucinating LLM. ITQE proves that hybrid verification is mandatory, but it also proves that human-in-the-loop oversight remains the ultimate safeguard.

Professional Relevance

This project demonstrates readiness for advanced AI engineering roles in the cybersecurity sector. It showcases the ability to move beyond single-model prototypes and engineer end-to-end systems that balance automation with safety. By integrating machine learning, generative AI, and agentic workflows, ITQE reflects the reality of modern production AI: systems must be bounded, auditable, and designed to fail safely.

The ITQE architecture—particularly the Slot Verifier—is directly applicable to real-world SOC platforms, regulatory compliance tooling, and any domain where LLMs are used to summarize high-stakes tabular data.

References

Alahmadi, B., Axon, L., & Martinovic, I. (2022). 99% false positives: A qualitative study of SOC analysts' perspectives on security alarms. In *Proceedings of the 31st USENIX Security Symposium* (pp. 2783–2800). USENIX Association.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys, 55*(12), 1–38. https://doi.org/10.1145/3571730

National Institute of Standards and Technology. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)* (NIST AI 100-1). U.S. Department of Commerce. https://doi.org/10.6028/NIST.AI.100-1

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. In *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications* (pp. 1–6). IEEE. https://doi.org/10.1109/CISDA.2009.5356528

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2210.03629